

GenAI Frameworks with LangChain

Chains • Prompt Templates • Context & Tools



CHAINS



MEMORY



TOOLS

Chains & Prompt Templates

WHAT'S AHEAD

Chains: composing multi-step LLM calls

Prompt templates: parameterized and reusable

Composing steps with LangGraph: state, nodes & edges

Branching a chain, and inspecting intermediate steps

One Prompt Can't Do a Two-Part Job

PROBLEM

- A real Sprint 3 task: classify a guest's request, then draft a fitting reply
- Cramming "classify AND respond" into one prompt does one half well, the other sloppily
- Or the prompt grows long and multi-purposed, and gets hard to test
- A single test case now has to check two different things about one answer at once

*"Classify this guest message
AND draft a reply to it..."*

one prompt, two jobs

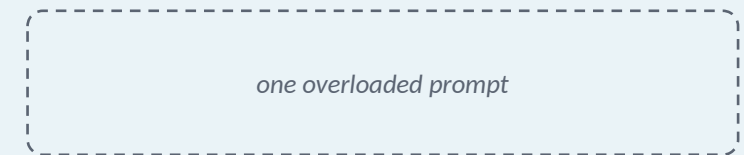
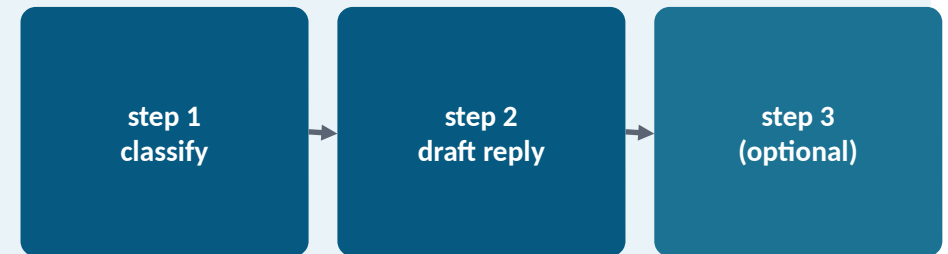
?

One oversized, overloaded prompt trying to do two different jobs at once

Chains: Composing Multiple LLM Calls

SOLUTION

- A chain: a deliberate sequence of steps, at least one calling an LLM
- A later step's input includes an earlier step's output
- "Chain" is a pattern name — LangChain's own old Chain classes moved to legacy langchain-classic
- A framework's real job: the plumbing between calls, not making the first call possible



An assembly line of small steps vs. one overloaded station

HANDS-ON Build a two-step chain that calls the LLM more than once to complete a task.

Retyping the Same Tested Prompt Doesn't Scale

PROBLEM

- A guardrailed, schema-tested prompt is worth reusing exactly as tested
- String-concatenating a variable into an f-string works for a five-minute script
- It doesn't scale once several steps each need their own carefully worded prompt
- Fixing wording in one place should mean every call site picks up the fix

"Classify: {msg}"

≠

copy 1

"Classify this: {msg}"

≠

copy 2

"Please classify {msg}"

≠

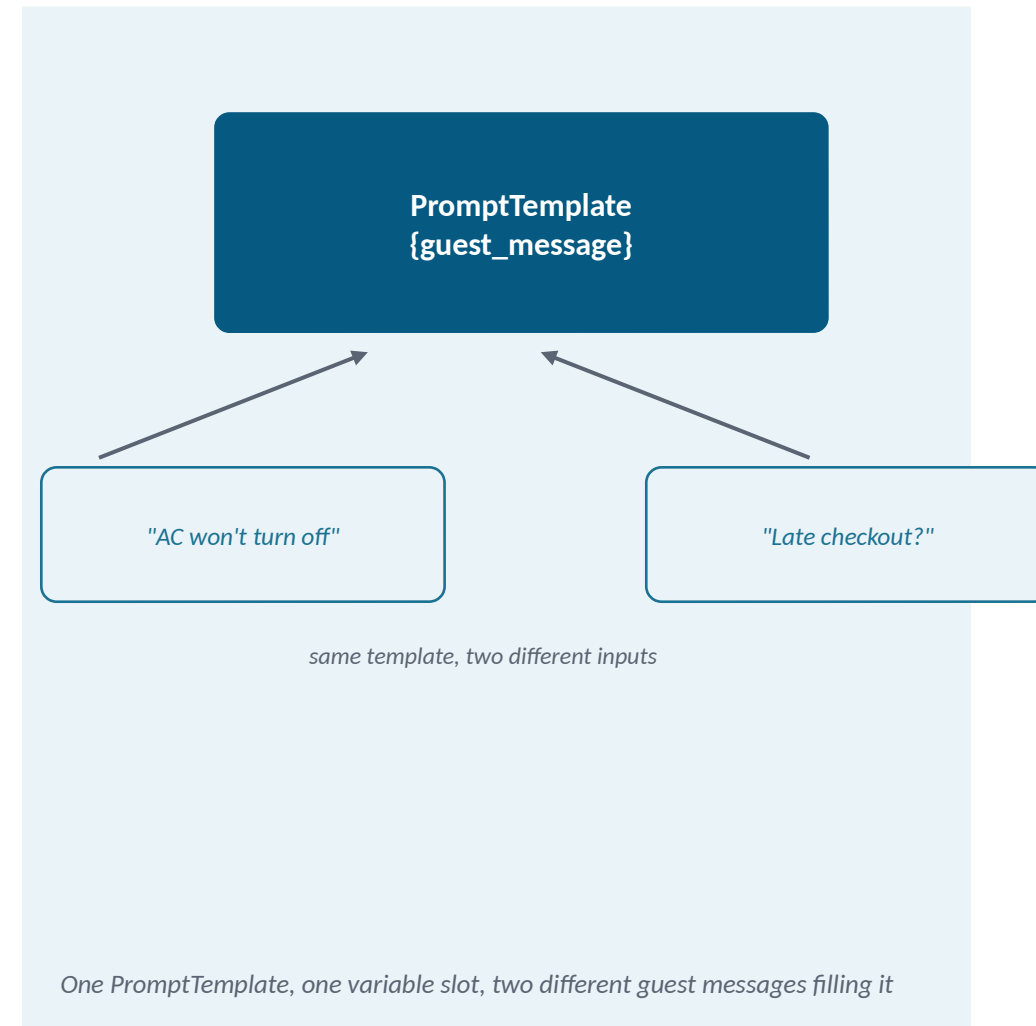
copy 3

Three near-identical, hand-copied prompts, each drifting a little from the tested wording

Prompt Templates: Parameterized, Reusable Prompts

SOLUTION

- PromptTemplate / ChatPromptTemplate, from langchain_core.prompts
- Fixed wording stored once, with named variable slots
- .invoke() / .format() fills the slots — the template's own wording never changes
- Supports f-string, mustache, or jinja2 — prefer f-string; untrusted jinja2 can execute code

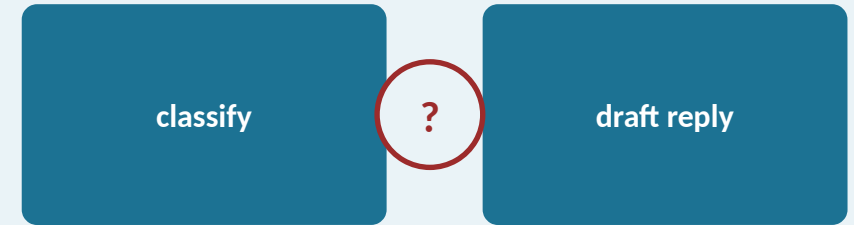


HANDS-ON Parameterize a prompt template and reuse it across two different inputs.

Two Calls, No Pipeline, No Way to Debug

PROBLEM

- A chain definition and a reusable template still don't run as one pipeline
- Nothing yet ties a first step's output into a second step's input
- If the final answer is wrong, there's no way to see what an earlier step produced
- That visibility is exactly what a facilitator, or a team, needs to debug it



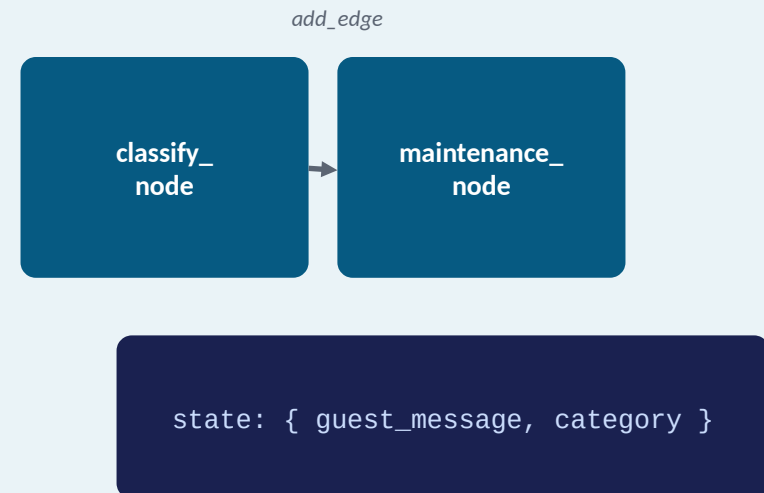
can't see step 1's output

Two boxes, no connecting pipe, and no visibility into either step's output

Composing Steps with LangGraph

SOLUTION

- State: a typed structure (TypedDict) carrying data between steps
- Node: a plain function — one step, reads state, may call a model, returns an update
- add_edge: a fixed connection, one node to the next — sequential composition
- StateGraph(...).compile() turns the assembled nodes and edges into a runnable chain



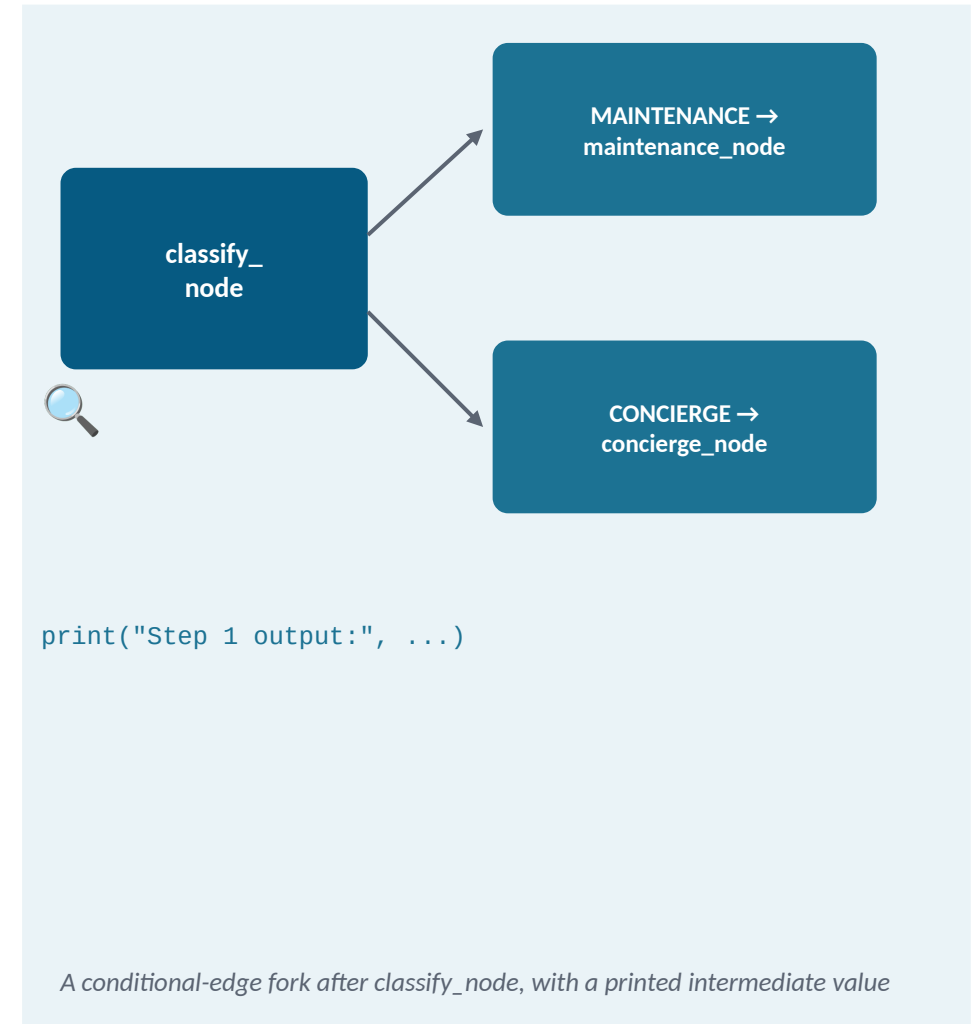
Two named nodes, connected by add_edge, sharing one typed state object

HANDS-ON Build a two-step chain that calls the LLM more than once to complete a task.

Branching & Inspecting Intermediate Steps

- `add_conditional_edges`: a routing function inspects state, picks the next node
- This is the branch — route to a different prompt template based on the input
- A node is a plain function: call it directly, or print inside it, to see its output
- No separate observability tool needed to debug a step in isolation

HANDS-ON Add a branch to the chain that routes to a different prompt template based on the input, and print each step's intermediate output to debug it.



Context, Memory & Tool Integration

WHAT'S AHEAD

Why a chain needs to remember earlier turns

From legacy memory classes to LangGraph checkpoints

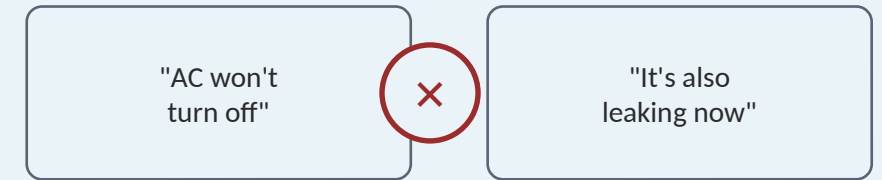
Tool integration: calling real functions from a chain

Starting your Sprint 3 AI feature

The Chain Forgets Everything Between Calls

PROBLEM

- The chain built so far is stateless between separate calls
- Call it once, get an answer — call it again, and step 1 has no idea a call already happened
- Several Sprint 3 briefs are explicitly multi-turn: a concierge chat, a follow-up detail
- Without deliberate state, a second message loses everything the first one established



1 minute later — no memory of the first message

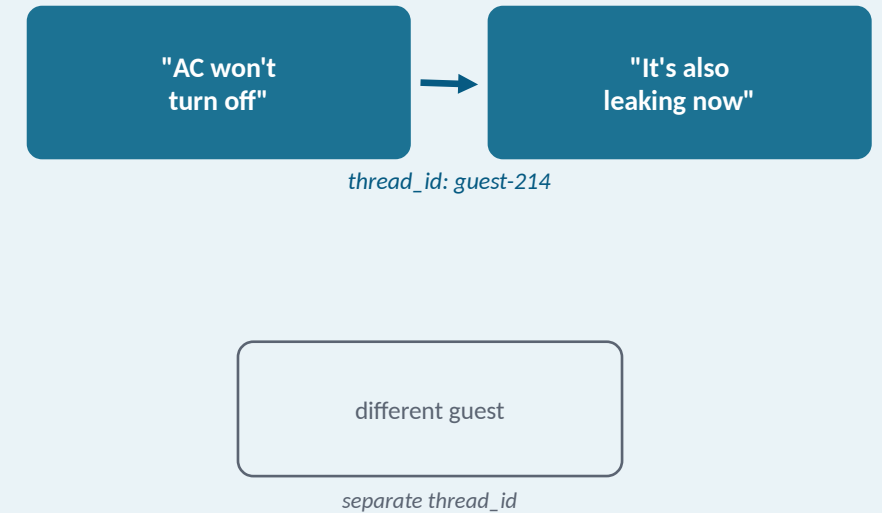
Two messages from the same guest, a minute apart — the second arrives with no memory of the first

Persistent Context with LangGraph Checkpointers

SOLUTION

- Legacy ConversationBufferMemory-style classes also moved to langchain-classic
- Current approach: a checkpointer, attached when the graph is compiled
- Every call with the same thread_id reads back, then updates, that thread's own state
- A different thread_id gets a fully separate history — one guest never sees another's

HANDS-ON Add persistent multi-turn context to the chain using LangChain's current recommended state approach.

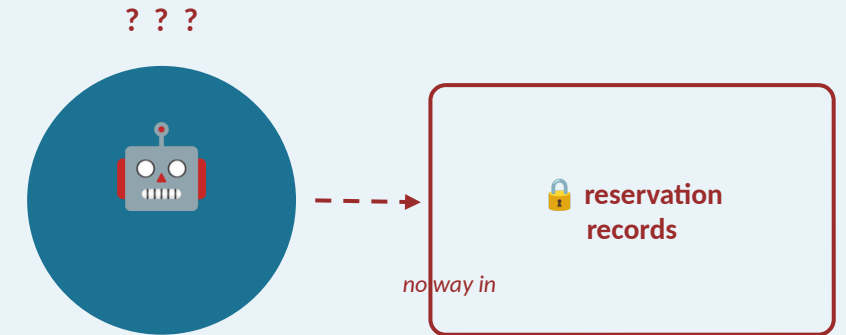


Same thread_id links two messages; a different thread_id keeps a separate guest's history apart

The Model Can't See Real Guest Data

PROBLEM

- A model only knows its training data and whatever text is in front of it
- One team's agent task: "check eligibility against the reservation record"
- Another's: actually create or update a real application record
- Both need real, live, external data and action — no prompt alone can produce either

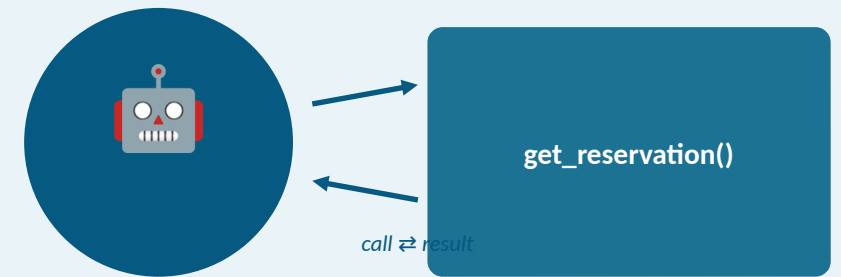


A model that has never seen this guest's reservation, with no way to reach it

Tool Integration: Calling Real Functions from a Chain

SOLUTION

- `@tool` decorator turns a plain Python function into something a model can call
- Type hints are mandatory — they define the tool's input schema
- The docstring becomes the tool's description — how the model knows when to use it
- `bind_tools([...])` + a `ToolNode` actually executes the call and returns the result



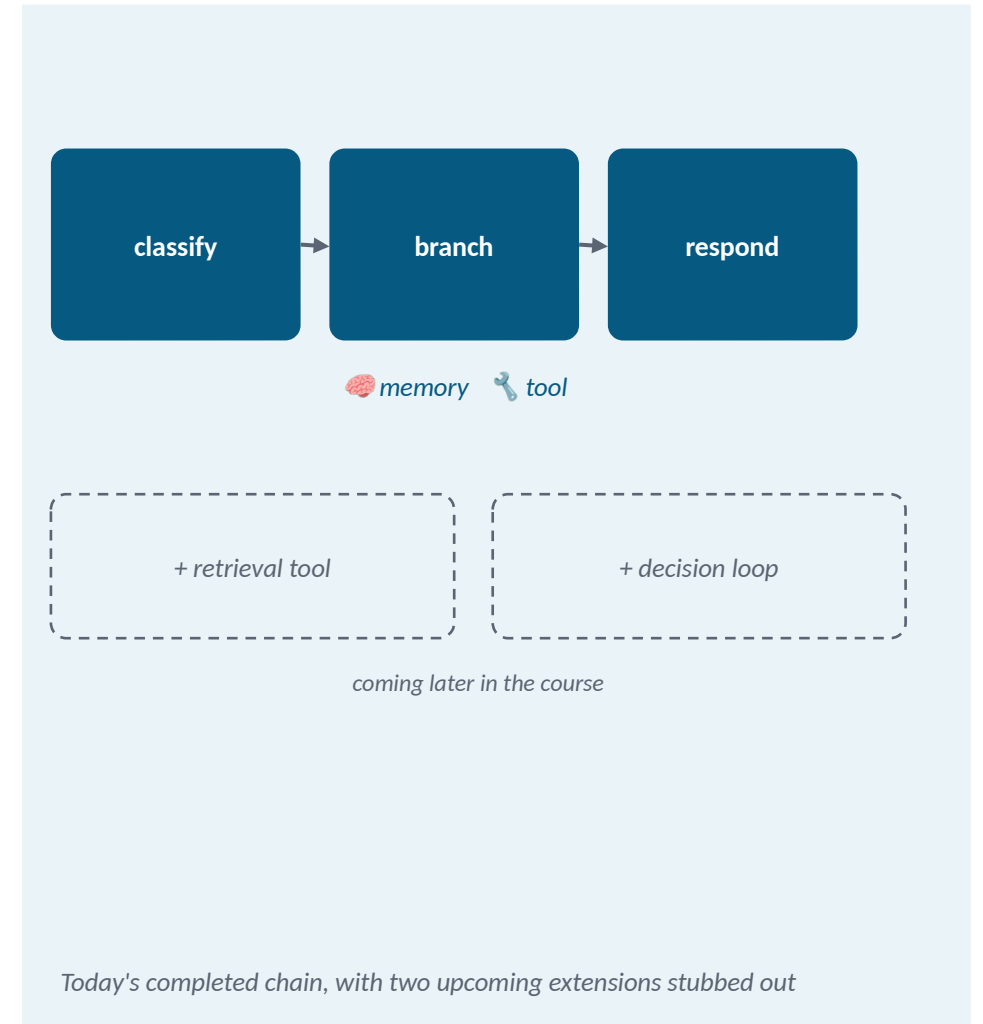
```
check_in=2026-09-20, status=confirmed
```

HANDS-ON Give the chain a callable tool or function and verify it invokes the tool correctly.

A single verified tool call — the model requests it, ToolNode executes it, the result comes back

Starting Your Sprint 3 AI Feature

- Today's chain: classifies, branches, remembers, and calls one verified tool
- Compare it against your own Team Brief's actual Sprint 3 AI feature
- Today's category names and tool are placeholders — swap in your team's real ones
- What's added next: a retrieval tool, then a bounded decision loop, later in the course



HANDS-ON Begin this sprint's capstone AI feature using this chain as its base.